IT585 – Advanced Machine Learning

Lab 2: Empirical Study of Hoeffding and VC-Dimension Bounds

General Instructions

1. All work must be done in a **single Google Colab notebook**
2. You may use numpy, scipy, matplotlib, and sklearn unless stated otherwise
3. Every plot **must be followed by a written explanation**
4. Do **not** hard-code results or reuse plots across sections
5. Your notebook must run end-to-end without errors
6. Deadline : February 17, 2026 11:59 AM IST
7. Name of your file should be : yourrollno_lab2.ipynb

Part A — Hoeffding Inequality Under Controlled Noise

Goal : Empirically verify Hoeffding's inequality and observe how it behaves under increasing label noise.

Step A1: Generate Data

- Input: $X \sim \mathcal{N}(0,1)$
- True labeling rule:
  $[Y = \mathbb{I}(X > 0)]$
- Labels are flipped independently with probability η

Code Snippet: Data Generation

```python
import numpy as np
```

```
def generate_data(n, noise=0.0, seed=None):
    if seed is not None:
        np.random.seed(seed)

    X = np.random.randn(n)
    Y = (X > 0).astype(int)

    # Flip labels with probability noise
    flips = np.random.rand(n) < noise
    Y[flips] = 1 - Y[flips]

    return X, Y
```

⚠️ **Do not modify this function across experiments.**

Step A2: Fixed Hypothesis

Use the fixed classifier:

$$[h(x) = \mathbb{I}(x > 0)]$$

Code Snippet

```python
def classifier(x):
    return (x > 0).astype(int)
```

Step A3: Experimental Procedure

For each noise level

$$\eta \in \{0, 0.1, 0.2, 0.4\}$$

and for each sample size

$$n \in \{10, 20, 50, 100, 500, 1000\}$$

do the following:

Generate a training dataset of size (n)

1. Compute **empirical error** on training data
2. Estimate **true error** using a very large test set (≥ 100,000 samples)
3. Repeat steps 1–3 at least **50 times**
4. Average the results

Step A4: Hoeffding Bound

$$\text{For confidence } (\delta = 0.05), \text{compute :}$$

$$\epsilon(n) = \sqrt{\frac{\log(2/\delta)}{2n}}$$

$$\text{Upper bound on true error:}$$

$$\hat{L}(h) + \epsilon(n)$$

Required Plots

1. Empirical vs true error (y-axis) vs (n)
2. Width of Hoeffding bound vs (n)
3. Separate plots for each noise level

Answer the questions:

- Why does the bound shrink with (n)?
- Why does noise **not** appear in Hoeffding's bound?

- Is the bound tight or loose in practice?

Part B — Vacuity of Hoeffding Bounds

Goal: Understand when a bound is mathematically correct but practically useless.

Step B1: Fix Parameters

- Confidence:

$$\delta = 0.05$$

- Use the same classifier as Part A

Step B2: Procedure

$$\begin{aligned}
&\text{For each } n \in \{10, 20, 50, 100\} \\
&\quad \text{Compute empirical error}(\hat{L}) \\
&\quad\quad \text{Compute Hoeffding} : \epsilon n \\
&\quad \text{Check whether: } [\hat{L} + \epsilon \geq 1]
\end{aligned}$$

Answer the Questions below

- At which (n) does the bound become non-vacuous?
- Does vacuity imply failure of learning?
- Why is Hoeffding pessimistic for small (n)?

Part C — VC Dimension and Hypothesis Complexity

Goal: See how generalization depends on **hypothesis class size**, not just sample size.

Hypothesis Classes

Class	Description	VC Dimension
$\mathcal{H}_1$	Single threshold	1
$\mathcal{H}_k$	Union of k intervals	$2k$

A k-interval classifier predicts

$$h(x) = 1 \quad \text{if } x \in \bigcup_{i=1}^{k} [a_i, b_i],$$

and 0 otherwise.

Step C1: Representing the Hypothesis

Use a **list of intervals**:

```python
intervals = [(a1, b1), (a2, b2), ..., (ak, bk)]
```

Step C2: Prediction Function

```python
def k_interval_predict(x, intervals):
    for a, b in intervals:
        if a <= x <= b:
            return 1
    return 0
```

Step C3: Empirical Risk Minimization (Key Part)

To find the best (k)-interval classifier:

1. **Sort data by input value**
2. Consider candidate intervals only between data points
3. Use a **greedy or dynamic programming approach**:
    - Each interval captures a contiguous block of positive labels
4. Choose up to (k) disjoint intervals that minimize training error

> Hint: For small (k), brute force over interval endpoints is acceptable.

Implement

- Your own interval selection logic
- No sklearn classifiers allowed here

Step C4: VC Generalization Bound

The VC generalization bound is

$$L(h) \leq \hat{L}(h) + \sqrt{\frac{d\left(\log\left(\frac{2n}{d}\right) + 1\right) + \log\left(\frac{4}{\delta}\right)}{n}}, \quad d = 2k.$$

Plots

1. Training error vs (k)
2. Test error vs (k)
3. VC bound width vs (k)

Answer the Questions:

- Why does training error decrease with (k)?
- Why does test error stop improving?
- How does the VC bound reflect this behavior?

Part D — Overfitting Without Noise

Goal: Show that overfitting can happen even with **perfect labels**.

Procedure

1. Generate **noise-free data**
2. Increase (k)
3. Track:
    - Training error
    - Test error
    - VC bound

Answer the Question

Why does overfitting occur even when labels are correct?

Part E — Bound Comparison

Tasks

For the same dataset:

1. Compute Hoeffding bound
2. Compute VC bound
3. Compare:
    - Sensitivity to (n)
    - Sensitivity to hypothesis class
    - Practical usefulness

Give your Insights by answering the question

Which bound would you trust more when designing a learning algorithm, and why?